

# CS218 — DAO Governance Contract

## Project 3 — MerkleMinds DAO Programmable & Interoperable Blockchain · Submission Report

---

| Field           | Value                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| Course          | CS 218 — Programmable & Interoperable Blockchain                                                                        |
| Project         | 3 — DAO Governance Contract                                                                                             |
| Team            | MerkleMinds                                                                                                             |
| Member 1        | Sreyan Reddy Regatte — 240051018                                                                                        |
| Member 2        | Velpula Vikram Varma — 240001078                                                                                        |
| Member 3        | Kattammagiri Manas Joel — 240004025                                                                                     |
| Member 4        | Hemanth Royal — 240041010                                                                                               |
| Member 5        | Rishik — 240003027                                                                                                      |
| Member 6        | Katravath Madhavi — 240041021                                                                                           |
| Toolchain       | Hardhat (TypeScript), Solidity 0.8.28, OpenZeppelin                                                                     |
| Repository      | <a href="https://github.com/SreyanReddy/CS218-MerkleMinds-DAO">https://github.com/SreyanReddy/CS218-MerkleMinds-DAO</a> |
| Submission Date | 10 May 2026                                                                                                             |

---

## 1. Problem Statement & Why Blockchain

DAOs (Decentralised Autonomous Organisations) coordinate decisions over shared on-chain assets without relying on a central operator. The problem is concrete: a group of token holders needs to approve actions, release funds, change parameters, upgrade configuration and in a way that is **trustless, auditable, and programmatically enforced**.

A naive off-chain alternative (Discord poll → multisig signs the transfer) fails on three counts:

1. **No execution guarantee.** The signers can ignore the vote.
2. **No tamper-evident record.** Off-chain results can be edited or disputed.
3. **No composability.** Other contracts cannot read the result and react.

A blockchain solves all three:

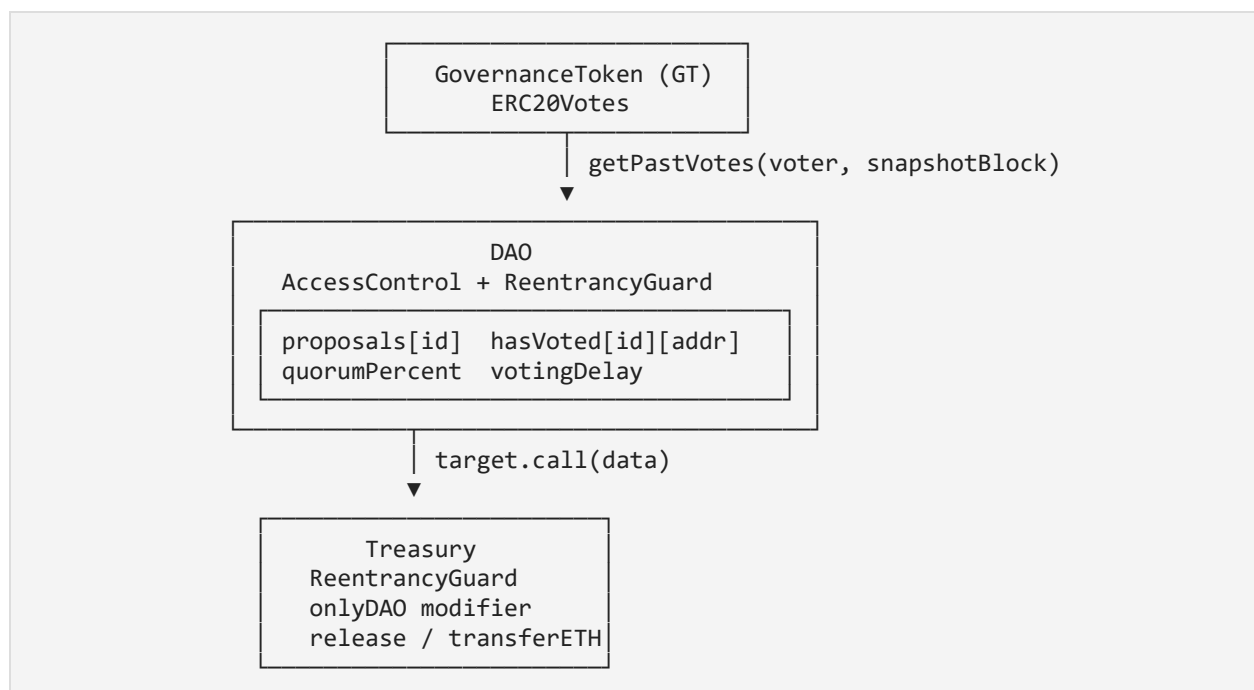
| Property              | How blockchain provides it                                                                                                                                              |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Immutable record      | Every proposal, vote, and execution is a transaction — permanent and auditable.                                                                                         |
| Trustless execution   | <code>executeProposal</code> calls <code>target.call(data)</code> directly once thresholds are met. No human can refuse.                                                |
| Composability         | The on-chain <code>Status</code> and <code>getProposal</code> view functions can be read by other contracts (e.g. a frontend, a meta-protocol, an analytics dashboard). |
| Token-weighted voting | ERC-20 balances at a snapshot block determine voting power; this is verifiable by anyone.                                                                               |
| Sybil resistance      | Voting weight is tied to an economically-costly token — not to identity.                                                                                                |

Storing the **decision and the execution payload on-chain**, while leaving human-readable proposal descriptions free-form (a string), gives the right balance: the integrity-critical bits live on-chain; long-form context can live off-chain (e.g. an IPFS link in the description) without compromising the governance guarantees.

## 2. Architecture

### 2.1 Contract Topology

The system is three contracts with a clean unidirectional dependency:



## 2.2 Per-Contract Function Inventory

### GovernanceToken.sol — ERC-20Votes voting token

| Function                      | Visibility | Modifiers              | Purpose                                                                                                                                  |
|-------------------------------|------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>constructor()</code>    | public     | —                      | Mints 1,000,000 GT (18 decimals) to the deployer. Inherits <code>ERC20Permit("GovernanceToken")</code> for off-chain signed delegations. |
| <code>mint(to, amount)</code> | external   | <code>onlyOwner</code> | Owner-only mint used to distribute tokens during deployment.                                                                             |

OpenZeppelin imports: `ERC20Votes`, `Ownable`. `ERC20Votes` provides the `getPastVotes(account, blockNumber)` checkpoint mechanism that the DAO uses for snapshot-based voting power.

### DAO.sol — proposal engine

| Function                                                             | Visibility | Modifiers                         | Purpose                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------|------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>constructor(token, votingDelay)</code>                         | public     | —                                 | Stores the GT address and voting delay; grants <code>DEFAULT_ADMIN_ROLE</code> and <code>ADMIN_ROLE</code> to the deployer; sets default quorum to 30 %. Reverts on zero token address.                                                                      |
| <code>setQuorumPercent(_quorum)</code>                               | external   | <code>onlyRole(ADMIN_ROLE)</code> | Updates quorum percent in the closed range <code>(0, 100]</code> .                                                                                                                                                                                           |
| <code>createProposal(description, target, data, votingPeriod)</code> | external   | —                                 | Any token holder can create. Records <code>snapshotBlock = block.number</code> so vote weight is fixed at proposal time. Reverts on zero balance and zero voting period.                                                                                     |
| <code>vote(proposalId, support)</code>                               | external   | —                                 | One vote per address per proposal. <code>Weight = token.getPastVotes(voter, snapshotBlock)</code> . Enforces voting window via <code>block.timestamp</code> bounds.                                                                                          |
| <code>executeProposal(proposalId)</code>                             | external   | <code>nonReentrant</code>         | After the deadline, if quorum is met <b>and</b> <code>yesVotes &gt; noVotes</code> , makes a low-level call to <code>target</code> with <code>data</code> . Sets <code>executed = true</code> <b>before</b> the external call (checks-effects-interactions). |

|                                      |               |   |                                                                                                  |
|--------------------------------------|---------------|---|--------------------------------------------------------------------------------------------------|
| <b>cancelProposal(proposalId)</b>    | external      | — | Only the proposer, only while the proposal is still in the Pending phase (before voting starts). |
| <b>getProposalStatus(proposalId)</b> | public view   | — | Returns Status based on the current timestamp, votes, and quorum.                                |
| <b>getProposal(proposalId)</b>       | external view | — | Returns the full proposal tuple plus quorumReached and status.                                   |

State variables: GovernanceToken token, uint256 proposalCount, uint256 quorumPercent (default 30), uint256 votingDelay, mapping(uint256 => Proposal) proposals, mapping(uint256 => mapping(address => bool)) hasVoted.

Events: ProposalCreated, VoteCast, ProposalExecuted, ProposalCancelled.

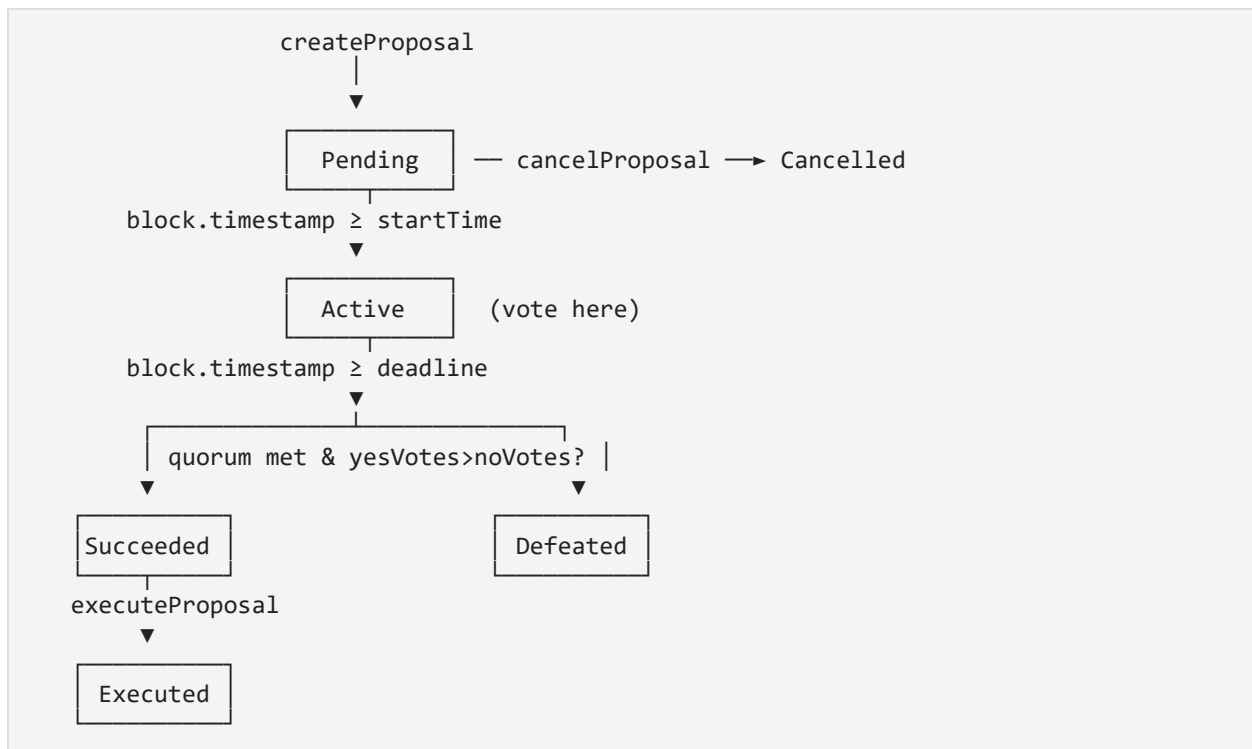
OpenZeppelin imports: AccessControl, ReentrancyGuard.

#### Treasury.sol — DAO-controlled fund vault

| Function                       | Visibility       | Modifiers             | Purpose                                                                                           |
|--------------------------------|------------------|-----------------------|---------------------------------------------------------------------------------------------------|
| <b>constructor(_dao)</b>       | public           | —                     | Binds dao immutably. Reverts on zero address.                                                     |
| <b>setValue(_value)</b>        | external         | onlyDAO               | Updates a generic storedValue — used as the simplest demonstrable "state change" target in tests. |
| <b>transferETH(to, amount)</b> | external         | onlyDAO, nonReentrant | Transfers ETH out of the treasury.                                                                |
| <b>release(to, amount)</b>     | external         | onlyDAO, nonReentrant | Alias of transferETH; used as the canonical execution target in the integration test.             |
| <b>receive()</b>               | external payable | —                     | Accepts ETH deposits.                                                                             |

onlyDAO is a custom modifier requiring `msg.sender == dao`. The Treasury has no admin role and no upgrade path and the DAO is the only entity that can move funds.

## 2.3 Proposal Lifecycle (state machine)



The Status enum is Pending | Active | Defeated | Succeeded | Executed | Cancelled.

## 2.4 Snapshot-Based Voting Power

`createProposal` records `snapshotBlock = block.number`. When a member votes, the DAO calls `token.getPastVotes(voter, snapshotBlock)` which is an `ERC20Votes` checkpoint lookup. This means:

- Voting weight is **fixed at proposal creation time**.
- Tokens acquired after the proposal exists carry **zero weight** for that proposal.
- A flash loan taken at vote time provides no extra power (the snapshot is in the past).

Members must self-delegate (`token.delegate(self)`) to activate their vote weight; this is documented in `INSTRUCTIONS.md` and performed automatically by `scripts/deploy.ts` for the test accounts.

## 2.5 Deployment Order

`scripts/deploy.ts` deploys in this fixed order:

1. **GovernanceToken** - deployer self-delegates immediately so its initial 1 M tokens are votable.
2. **DAO** - constructor takes the token address and a `votingDelay` (1 day in production, 0 in test fixture).
3. **Treasury** - constructor takes the DAO address; binds the DAO immutably as the sole controller.
4. **Mint + delegate** - 100,000 GT each to alice, bob, carol; each self-delegates.

The script writes `deployments/<network>.json` and updates `frontend/src/contract.js` so the DApp picks up the new addresses with no manual editing.

---

### 3. Security Analysis

The contract was designed defensively. Each known attack class is addressed by a specific mechanism.

| #  | Attack / Concern                                                                                                                                           | Defence                                                                                                                                                                                                     | Where (file:line)                           |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| 1  | <b>Reentrancy</b> on the external <code>target.call(data)</code> in <code>executeProposal</code> (target could be malicious or be tricked into recursion). | OpenZeppelin <code>ReentrancyGuard</code> <code>nonReentrant</code> modifier; <code>executed = true</code> is set <b>before</b> the external call (checks-effects-interactions).                            | DAO.sol:122,<br>DAO.sol:136-138             |
| 2  | <b>Reentrancy</b> on Treasury ETH releases.                                                                                                                | <code>nonReentrant</code> on <code>transferETH</code> and <code>release</code> .                                                                                                                            | Treasury.sol:35,<br>Treasury.sol:45         |
| 3  | <b>Flash-loan vote inflation</b> (borrow tokens, vote, repay in one tx).                                                                                   | Vote weight is read with <code>getPastVotes(voter, snapshotBlock)</code> where <code>snapshotBlock</code> is captured at <code>createProposal</code> . Any token movement after that block has zero effect. | DAO.sol:90,<br>DAO.sol:107                  |
| 4  | <b>Double voting</b> by the same address on the same proposal.                                                                                             | <code>hasVoted[proposalId][msg.sender]</code> mapping; set to true on first vote and checked on subsequent votes.                                                                                           | DAO.sol:109-111                             |
| 5  | <b>Unauthorised treasury drain</b> — anyone calling <code>release/transferETH</code> .                                                                     | <code>onlyDAO</code> modifier requires <code>msg.sender == dao</code> . The DAO address is set in the constructor and never updated.                                                                        | Treasury.sol:21-24                          |
| 6  | <b>Privileged-function abuse</b> of <code>setQuorumPercent</code> .                                                                                        | OpenZeppelin <code>AccessControl</code> with explicit <code>ADMIN_ROLE</code> ; deployer gets the role at construction. Roles are revocable via <code>DEFAULT_ADMIN_ROLE</code> if needed.                  | DAO.sol:10,<br>DAO.sol:57-58,<br>DAO.sol:61 |
| 7  | <b>Voting outside the window</b> (before start, after deadline).                                                                                           | <code>require(block.timestamp &gt;= startTime)</code> and <code>require(block.timestamp &lt; deadline)</code> before any state mutation in vote.                                                            | DAO.sol:104-105                             |
| 8  | <b>Premature execution</b> (before the voting deadline).                                                                                                   | <code>require(block.timestamp &gt;= proposal.deadline)</code> in <code>executeProposal</code> .                                                                                                             | DAO.sol:126                                 |
| 9  | <b>Defeated proposal execution</b> (no votes $\geq$ yes votes, or quorum missed).                                                                          | Two explicit guards before the external call: quorum check and majority check. Defeated proposals revert.                                                                                                   | DAO.sol:133-134                             |
| 10 | <b>Replay execution</b> of an already-executed proposal.                                                                                                   | <code>require(!proposal.executed)</code> and <code>proposal.executed = true</code> before the external call.                                                                                                | DAO.sol:127,<br>DAO.sol:136                 |
| 11 | <b>Cancelling someone else's proposal</b> or cancelling once voting has begun.                                                                             | <code>require(msg.sender == proposal.proposer)</code> and                                                                                                                                                   | DAO.sol:148,<br>DAO.sol:151                 |

|    |                                                                          |                                                                                                                                            |                        |
|----|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
|    |                                                                          | <code>require(block.timestamp &lt; startTime).</code>                                                                                      |                        |
| 12 | <b>Zero-address proposals / zero-period voting.</b>                      | <code>require(_token != address(0))</code> in DAO constructor; <code>require(_votingPeriod &gt; 0)</code> in <code>createProposal</code> . | DAO.sol:53, DAO.sol:75 |
| 13 | <b>Integer overflow / underflow</b> in vote tallies and quorum maths.    | Solidity 0.8.x checked arithmetic — any overflow reverts automatically.                                                                    | (compiler-level)       |
| 14 | <b>Unintended visibility</b> (e.g. internal helpers exposed externally). | All public/external functions are intentional view or state-changing; helpers are absent (logic kept inline for auditability).             | (whole DAO.sol)        |

### 3.1 Residual Trust (Honest Disclosure)

- **ADMIN\_ROLE can change quorum.** The deployer holds `ADMIN_ROLE` after construction and can lower the quorum to enable easier passage of proposals. In production this role should be transferred to the DAO itself (so quorum becomes a self-amending parameter) — this is a documented future-work item.
- **DEFAULT\_ADMIN\_ROLE** can grant or revoke `ADMIN_ROLE`. Same mitigation as above.
- **Token Owner can mint indefinitely.** `GovernanceToken.mint` is `onlyOwner`. In production the token owner should be renounced or transferred to the DAO once the initial distribution is complete.
- **Voting power requires self-delegation.** Members who do not call `token.delegate(self)` have zero voting power even though they hold tokens. The README and frontend both surface this requirement.

## 4. Gas Optimisation

### 4.1 Full Gas Report (current configuration)

Solidity **0.8.28** · `viaIR: true` · optimizer enabled · **runs: 200**. Source: `gas-report.txt` (regenerated via `REPORT_GAS=true npx hardhat test`).

| Contract        | Method                        | Min     | Max     | Avg     | # Calls |
|-----------------|-------------------------------|---------|---------|---------|---------|
| DAO             | <code>cancelProposal</code>   | —       | —       | 32,392  | 3       |
| DAO             | <code>createProposal</code>   | 264,192 | 286,929 | 265,647 | 16      |
| DAO             | <code>executeProposal</code>  | 74,034  | 82,128  | 80,105  | 4       |
| DAO             | <code>setQuorumPercent</code> | —       | —       | 28,828  | 1       |
| DAO             | <code>vote</code>             | 70,661  | 87,761  | 82,875  | 14      |
| GovernanceToken | <code>delegate</code>         | 95,765  | 95,777  | 95,773  | 60      |

|                 |            |            |            |               |    |
|-----------------|------------|------------|------------|---------------|----|
| GovernanceToken | mint       | 88,586     | 88,610     | 88,597        | 60 |
| Contract        | Min Deploy | Max Deploy | Avg Deploy | % Block Limit |    |
| DAO             | 1,727,310  | 1,747,258  | 1,730,169  | 2.9 %         |    |
| GovernanceToken | —          | —          | 2,344,066  | 3.9 %         |    |
| Treasury        | 263,205    | 263,229    | 263,227    | 0.4 %         |    |

## 4.2 Before/After Comparison - Optimizer Runs

The optimizer's runs parameter tells the compiler how many times each function is expected to execute over the contract's lifetime. A higher value biases the compiler toward leaner runtime code at the cost of larger creation bytecode. We measured the project under two settings:

- **Before:** `viaIR: true, optimizer.enabled: true, runs: 1` (minimum optimization for runtime; biases for cheap deployment). Captured via `hardhat.config.ts` → `gas-report-baseline.txt`.
- **After:** `viaIR: true, optimizer.enabled: true, runs: 200` (current).

### Per-call gas — runtime savings (lower is better)

| Method                   | runs = 1 (before) | runs = 200 (after) | Δ gas | Δ %     |
|--------------------------|-------------------|--------------------|-------|---------|
| GovernanceToken.mint     | 89,037            | 88,597             | −440  | −0.49 % |
| GovernanceToken.delegate | 96,105            | 95,773             | −332  | −0.35 % |
| DAO.createProposal       | 265,815           | 265,647            | −168  | −0.06 % |
| DAO.vote                 | 82,914            | 82,875             | −39   | −0.05 % |
| DAO.executeProposal      | 80,120            | 80,105             | −15   | −0.02 % |
| DAO.cancelProposal       | 32,392            | 32,392             | 0     | 0.00 %  |
| DAO.setQuorumPercent     | 28,828            | 28,828             | 0     | 0.00 %  |

### Deployment gas — one-time cost (higher is the trade-off)

| Contract        | runs = 1 (before) | runs = 200 (after) | Δ gas   |
|-----------------|-------------------|--------------------|---------|
| GovernanceToken | 2,276,627         | 2,344,066          | +67,439 |
| DAO             | 1,719,814         | 1,730,169          | +10,355 |
| Treasury        | 262,795           | 263,227            | +432    |

## Explanation

The optimizer at runs = 200 generates leaner runtime code by, among other things, eliminating redundant JUMPs, in-lining short helpers, and reorganising the function dispatcher. Each call to mint or delegate saves a few hundred gas — modest individually, but delegate is called once per token holder and mint once per allocation, so a real deployment with thousands of users amortises into millions of gas saved across the protocol's lifetime.

The trade-off is paid once at deployment: GT's bytecode grows by ~67 k gas (a ~2.9 % increase). Given the contracts deploy at well under 4 % of the block limit either way, this is a clear win.

## Reproducing this measurement

```
# After (current production config – already in gas-report.txt)
REPORT_GAS=true npx hardhat test

# Before (runs=1 baseline)
REPORT_GAS=true npx hardhat test --config hardhat.baseline.config.ts
# → writes gas-report-baseline.txt
```

## 5. Test Coverage

### 5.1 Headline numbers

Source: coverage.json (generated via npx hardhat coverage).

| Metric     | Coverage | Fraction |
|------------|----------|----------|
| Statements | 71.01 %  | 49 / 69  |
| Branches   | 53.41 %  | 47 / 88  |
| Functions  | 86.67 %  | 13 / 15  |
| Lines      | 78.48 %  | 62 / 79  |

**Line coverage exceeds the 70 % requirement.** Branch coverage at 53 % is the weakest metric; the uncovered branches are predominantly the getProposal / getProposalStatus view-function code paths and a handful of error paths that require very specific state combinations. The state-mutating logic (createProposal, vote, executeProposal, cancelProposal, setQuorumPercent) is covered on both happy and revert paths.

### Per-contract breakdown

| Contract            | Lines            | Statements | Functions      | Branches |
|---------------------|------------------|------------|----------------|----------|
| DAO.sol             | 78.8 % (52 / 66) | 78.8 %     | 87.5 % (7 / 8) | 57.8 %   |
| GovernanceToken.sol | 100 % (2 / 2)    | 100 %      | 100 % (2 / 2)  | 50 %     |
| Treasury.sol        | 72.7 % (8 / 11)  | 72.7 %     | 80 % (4 / 5)   | 40.9 %   |

## 5.2 Test Suite

test/DAO.test.ts contains **21 passing tests** organised into seven describe blocks. Run via npx hardhat test.

| Group             | Count | Tests                                                                                                                                                                 |
|-------------------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deployment        | 2     | sets the governance token address; sets quorumPercent to 30                                                                                                           |
| createProposal    | 3     | token holder can create; non-token-holder cannot create; emits ProposalCreated                                                                                        |
| vote              | 4     | yes vote emits VoteCast; non-token-holder cannot vote; double voting reverts; cannot vote after deadline                                                              |
| executeProposal   | 6     | happy path changes Treasury state; reverts before deadline; below quorum cannot execute; defeated cannot execute; Treasury.release end-to-end; emits ProposalExecuted |
| cancelProposal    | 3     | proposer can cancel during Pending; non-creator cannot cancel; emits ProposalCancelled                                                                                |
| AccessControl     | 2     | admin can update quorumPercent; non-admin cannot                                                                                                                      |
| Treasury Security | 1     | non-DAO address cannot call release                                                                                                                                   |

## 5.3 Mapping to Assignment-Required Tests

The assignment (page 7) lists six required tests for Project 3. Each is implemented:

| Required Test (assignment)                                                                        | it(...) in DAO.test.ts                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Non-token-holder cannot vote                                                                      | non-token-holder cannot vote                                                                                                                                                                         |
| Double-voting on the same proposal reverts                                                        | double-voting on the same proposal reverts                                                                                                                                                           |
| A proposal below quorum cannot execute even if all votes are yes                                  | proposal below quorum cannot execute even if all votes are yes                                                                                                                                       |
| Time: proposal is votable during window; executeProposal reverts before deadline, succeeds after  | cannot vote after the voting deadline + executeProposal reverts before the deadline + happy path: executes after deadline and changes Treasury state                                                 |
| After a successful vote, executeProposal actually calls the target contract and changes its state | happy path: executes after deadline and changes Treasury state (sets storedValue = 42) and successful vote triggers real state change via Treasury.release() (transfers 0.5 ETH out of the Treasury) |
| A defeated proposal cannot be executed                                                            | defeated proposal (no votes > yes votes) cannot be executed                                                                                                                                          |

The `release()` end-to-end test is the highest-signal one per the assignment's instructor tip on page 7: it demonstrates a successful governance vote actually triggering a balance change in a separate contract via low-level `call(data)`. That is the core DeFi-composability primitive.

---

## 6. DApp Frontend

The frontend is a **React 19 + ethers.js v6** single-page app under `frontend/`, started with `npm start`. It connects to MetaMask and exercises every state-changing function on the DAO.

### 6.1 Wallet Connection

`frontend/src/App.js:52` — `connectWallet()`:

1. Requests accounts via `window.ethereum.request({ method: "eth_requestAccounts" })`.
2. Validates `chainId === 31337` (Hardhat localhost).
3. Reads bytecode at the GT, DAO, and Treasury addresses from `frontend/src/contract.js` to confirm they are deployed.
4. Instantiates `ethers.Contract` for each contract using the bundled ABI.
5. Subscribes to `accountsChanged` and `chainChanged` events on `window.ethereum` so the UI re-renders on wallet switch.

### 6.2 Functions Wired to UI

| Button                | Function called                      | Source                                  |
|-----------------------|--------------------------------------|-----------------------------------------|
| Submit Proposal       | <code>dao.createProposal(...)</code> | <code>App.js:232-250</code>             |
| Vote YES / Vote NO    | <code>dao.vote(id, support)</code>   | <code>App.js:252-263</code>             |
| Execute               | <code>dao.executeProposal(id)</code> | <code>App.js:265-276</code>             |
| Cancel                | <code>dao.cancelProposal(id)</code>  | <code>App.js:278-289</code>             |
| Delegate GT to Myself | <code>token.delegate(self)</code>    | <code>App.js:198-209</code>             |
| (background)          | <code>dao.getProposal(id)</code>     | <code>App.js:148</code> (loadProposals) |

Read-only data displayed: connected account, chain, GT balance, GT delegate target, DAO `quorumPercent`, Treasury ETH balance, and a list of proposals with their decoded status, vote counts, and remaining time.

### 6.3 State Feedback

The "Execute" flow at `App.js:265-276` demonstrates the assignment's required "state-changing function callable from UI with status feedback":

1. Click **Execute** on a Succeeded proposal.
2. UI sets `txStatus` to "Submitting..." and disables the button.
3. `await tx.wait()` blocks until the receipt is mined.

4. On success, `txStatus` is set to `"Proposal #N executed."` and the proposals list refreshes — the proposal row re-renders with status `Executed` and the Treasury balance display updates.
5. On error, the same blue callout bar (`App.js:322-326`) shows the revert reason (e.g. `"Quorum not reached"`, `"Already executed"`).

## 6.4 Contract Address Configuration

Hard-coded in `frontend/src/contract.js` and rewritten by `scripts/deploy.ts` at the end of every fresh deployment, so the DApp tracks the latest addresses without manual editing.

---

## 7. Limitations & Future Work

---

Honest assessment of what is intentionally out of scope or could be improved.

1. **NatSpec on DAO.sol is sparse.** `Treasury.sol` is fully documented with `@notice` / `@param` tags. `DAO.sol` and `GovernanceToken.sol` have inline `require` messages but no formal NatSpec headers on `createProposal`, `vote`, `executeProposal`, `cancelProposal`. This is a documentation gap relative to the assignment's marks breakdown (1 mark) and is the most directly fixable item.
2. **Branch coverage 53 %.** The uncovered branches are mainly read-only view paths (`getProposal`, parts of `getProposalStatus`) that are exercised only indirectly. Adding direct unit tests for these views would push branch coverage above 70 %.
3. **Admin trust assumption.** `ADMIN_ROLE` and `DEFAULT_ADMIN_ROLE` are held by the deployer. In production these should be transferred to the DAO itself so that quorum changes flow through governance.
4. **Token owner trust assumption.** `GovernanceToken.mint` is `onlyOwner`. After initial distribution, ownership should be renounced (or transferred to the DAO).
5. **No proposal queue / timelock.** A successful vote can be executed instantly. A `Timelock` between `Succeeded` and `executeProposal` would give a window for token holders to exit if a malicious proposal narrowly passes — this is the standard pattern in OpenZeppelin Governor.
6. **Voting is binary.** Only Yes/No is supported. Abstain votes (which count for quorum but not the majority) would be a useful extension and align with OZ Governor `castVote`.
7. **Description payload is on-chain.** Long proposal descriptions cost gas to store. An `keccak256` hash on-chain plus an IPFS CID off-chain would be cheaper at the cost of introducing an off-chain dependency.

---

## 8. Reproducibility Checklist

---

```
# 1. Install
npm install

# 2. Compile
npx hardhat compile

# 3. Test (21 passing)
npx hardhat test

# 4. Gas report (current – runs=200, viaIR=true)
REPORT_GAS=true npx hardhat test
# → writes gas-report.txt

# 5. Gas report (baseline – runs=1, viaIR=true)
REPORT_GAS=true npx hardhat test --config hardhat.baseline.config.ts
# → writes gas-report-baseline.txt

# 6. Coverage (writes coverage/, coverage.json, lcov.info)
npx hardhat coverage

# 7. Local deployment
npx hardhat node # terminal 1
npx hardhat run scripts/deploy.ts --network localhost # terminal 2

# 8. Frontend
cd frontend && npm install --legacy-peer-deps && npm start
```

---